THE ARCHITECT'S FIELD NOTES

BEHAVIORAL INTERVIEW STUDY GUIDE / 05 OF 07

Design 3 — Double-Entry Financial Ledger

THINK CLEARLY. DECIDE WITH EVIDENCE. DELIVER WITH CONFIDENCE.

Evidence-based STAR answers, senior-level judgment, and focused interview rehearsal.

PREPARED FOR SENIOR / PRINCIPAL SOLUTION ARCHITECT INTERVIEWS

READ · REFLECT · PRACTICE

05



YOUR STUDY COMPANION

10-minute system design answer I Solution Architect interview preparation

NAVIGATION

Inside this guide

01 1. Problem & Requirements

02 2. Understand Double-Entry Accounting

03 3. High-Level Architecture

04 4. Key Design Decisions — With Reasons

05 5. End-to-End Request Flow

06 6. Failure Handling, Scalability & Security

07 7. Key Points to Remember

08 8. Interview Closing Summary

CHAPTER

1. Problem & Requirements

Design a financial ledger that reliably records money movement and maintains correct account balances.

Functional requirements

- Record financial transactions using debit and credit entries.
- Maintain accurate balances.
- Support refunds, reversals, and adjustments.
- Prevent duplicate postings.
- Provide transaction history and audit records.

Non-functional requirements

- **Strong consistency:** Financial entries must be correct.
- **Atomicity:** All entries of a transaction succeed or fail together.
- **Immutability:** Posted financial records cannot be silently modified.
- **High availability and durability:** No loss of acknowledged financial postings.
- **Auditability:** Every change must be traceable.

Key assumption: We are designing the authoritative ledger for a financial product, not just a transaction-history API.

CHAPTER

2. Understand Double-Entry Accounting

Every financial transaction records both a debit and a credit.

Example: A $100 credit card purchase clears (simplified issuer-accounting perspective):

Account	Debit	Credit
Cardholder Receivable (Asset)	$100	—
Merchant Settlement Payable (Liability)	—	$100
Total	**$100**	**$100**

Core invariant: `Sum(debits) = Sum(credits)` for each posted journal transaction, separately for each currency.

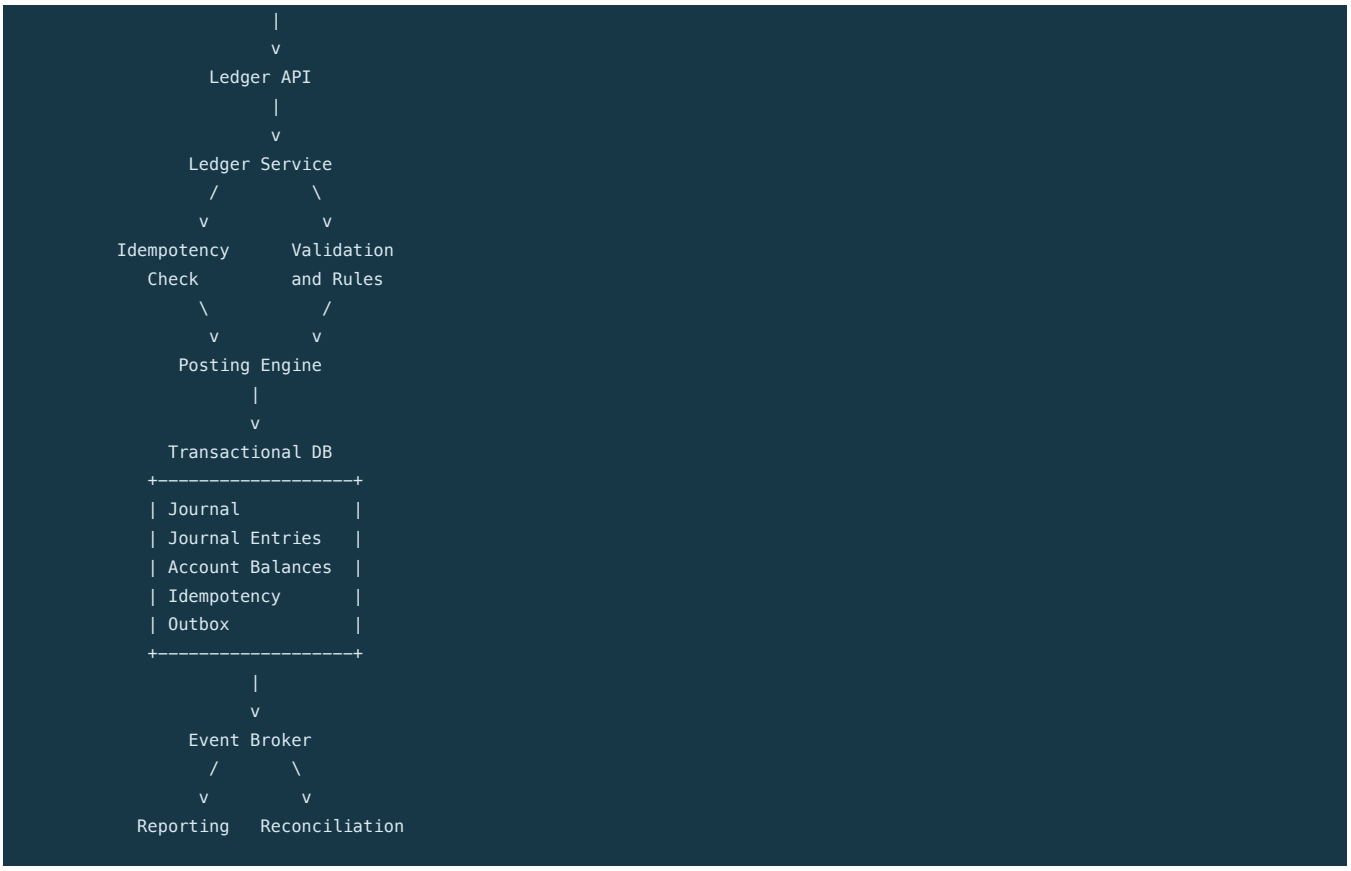
The actual accounting entries depend on the issuer, network, settlement model, and applicable accounting rules. Debit and credit do not simply mean money leaving or entering a bank account; their effect depends on account type.

Why not simply update a balance? A balance-only database may show the current amount without reliably preserving *why* it changed. A double-entry ledger stores the accounting effect and allows balances to be reconstructed and checked.

CHAPTER

3. High-Level Architecture

```
            Payment / Clearing Service
```



Why these components?

- **Ledger Service:** Centralizes accounting rules and posting.
- **Transactional DB:** Commits all related financial entries together.
- **Balance table:** Enables fast balance lookups.
- **Idempotency record:** Prevents the same business event from posting twice.
- **Outbox and event broker:** Distribute committed financial events to downstream systems.

The journal is the accounting source of truth; reporting and analytics are derived views.

CHAPTER

4. Key Design Decisions — With Reasons

Decision	Choice	Why / Trade-off
Database	Relational DB with ACID transactions	All journal entries must commit atomically; other transactional databases may also work.
Accounting model	Double-entry	Balanced, traceable postings.
History	Append-only posted entries	Preserve an audit trail; correct with new entries.
Balance reads	Transactionally maintained balance table	Fast reads without summing all entries.
Consistency	Strong for posting	No partial journal commits.
Idempotency	Unique business posting identity	Prevent duplicate postings on retries.

Messaging	Broker + transactional outbox	Reliable downstream delivery without blocking commits.
Scaling	Partition by accounting domain if needed	Keep journal postings atomic where possible.

Decision 1 — Why ACID Transactions?

```
BEGIN TRANSACTION
  Debit Receivable      $100
  Credit Payable        $100
  Update account balances
  Save idempotency record
  Save outbox event
COMMIT
```

If one required operation fails, the whole transaction rolls back. A debit without its matching credit must never commit.

Decision 2 — Why Immutable Journal Entries?

If a $100 posting is wrong, do **not** overwrite it with $80. Preserve the original and create reversing and corrected journal entries.

Why: Auditability, traceability, reconciliation, and the ability to reconstruct prior financial states.

Concept — append-only: Posted accounting entries are not changed or deleted; corrections are additional entries.

Decision 3 — How Do We Read Balances Quickly?

Option A: Calculate balances from all journal entries. This is reliable but may become expensive for large accounts.

Option B: Maintain a balance table together with each journal posting.

```
Ledger Posting
      |
      +--> Append journal entries
      |
      +--> Update account balances
      |
      v
Commit together
```

Choice: Option B. The journal remains the source of truth; balances are transactionally maintained projections that are independently checked against it.

Decision 4 — How Do We Prevent Duplicate Postings?

A clearing event can arrive more than once. Give the logical business posting a stable, properly scoped unique identity.

```
Receive posting
      |
      v
Already posted?
   /       \
 Yes        No
  |          |
  v          v
Return     Validate and
```

```
existing   post journal
result
```

The database unique constraint is the final protection against simultaneous duplicates.

Remember: Exactly-once business effect comes from durable idempotency, not a guarantee that the message is delivered only once.

Decision 5 — How Do We Handle Concurrency?

A prepaid account with $100 must not approve two simultaneous $80 withdrawals. Double-entry accounting by itself does not prevent overdrafts.

Use an **atomic balance condition** or appropriate row locks within the posting transaction. This enforces spending rules as well as balanced accounting.

Decision 6 — How Do We Scale?

First scale stateless API workers and optimize queries, indexes, connection pools, and database capacity.

Shard only when necessary. A journal entry may move value between accounts, so naive account-ID sharding can split one posting across databases. Prefer keeping related accounts in one accounting partition or using a database with the required cross-partition atomic transactions.

CHAPTER

5. End-to-End Request Flow

```
1. Receive financial posting request
              |
              v
2. Validate transaction and accounts
              |
              v
3. Check idempotency
              |
              v
4. Build debit and credit entries
              |
              v
5. Verify debits = credits
              |
              v
6. Begin ACID transaction
              |
              v
7. Insert journal entries
   Update balances
   Save idempotency + outbox
              |
              v
8. Commit transaction
              |
              v
9. Return posting confirmation
              |
              v
10. Publish events asynchronously
```

Critical: Balance checks, journal entries, and authoritative balance updates must be coordinated within one transaction where they enforce financial invariants.

For multi-currency transactions, define FX conversion and dedicated balancing/clearing accounts. A USD debit and EUR credit do not directly balance numerically.

CHAPTER

6. Failure Handling, Scalability & Security

Scenario	Design response	Why
Failure after first journal entry	Roll back entire transaction	Avoid partial posting.
Duplicate request	Durable idempotency	Avoid double posting.
Response lost after commit	Return stored result on retry	Preserve one business effect.
Broker unavailable	Transactional outbox	Retain event for later delivery.
Incorrect posting	New reversal/correction entries	Preserve the audit history.
Concurrent withdrawals	Atomic balance enforcement	Prevent invalid negative balances.
Database or region failure	Durable replication and fenced failover	Avoid lost/conflicting writes.
Balance mismatch	Reconcile against journal and external sources	Detect discrepancies.

Security and operations

- Encrypt sensitive data and restrict posting permissions.
- Separate ledger writers from reporting readers.
- Preserve tamper-evident audit trails.
- Test backups, restoration, and reconciliation.
- Define recovery point and recovery time objectives.
- Avoid independently writable regional replicas without correct coordination.

Concept — reconciliation: Comparing financial records or balances with another authoritative source to detect missing, duplicated, incorrect, or unmatched transactions.

Important: A balanced journal can still be *economically wrong*. Equal debit and credit totals are necessary, not sufficient, for correctness.

CHAPTER

7. Key Points to Remember

1. **Double-entry:** For each posted journal, total debits equal total credits.
2. **ACID:** Entries and authoritative balance updates commit together.
3. **Immutability:** Do not rewrite posted entries; reverse and correct them.
4. **Idempotency:** Retries must not create duplicate financial postings.
5. **Source of truth:** The journal is authoritative; materialized balances improve performance and must be reconciled.

CHAPTER

8. Interview Closing Summary

> I would design the financial ledger using double-entry accounting backed by a transactional database. Every posting has balanced debit and credit entries committed atomically with balance updates. Posted entries are immutable, and corrections use reversing entries. Durable idempotency prevents duplicate postings. I would maintain materialized balances for fast reads and use a transactional outbox for downstream events. For scale, I would partition carefully without breaking journal atomicity. Reconciliation, strict access controls, and tested recovery protect financial correctness and auditability.

Connection to Design 2: Authorization

Authorization system	Financial ledger
Decides whether to approve a purchase.	Records financial/accounting effects.
Creates an authorization hold.	Creates balanced journal entries.
Prioritizes low-latency risk and credit checks.	Prioritizes posting integrity and auditability.
Handles authorization reversal/expiration.	Handles financial reversal/correction.

An authorization hold is **not automatically** a posted ledger entry.

Next design: Payment Clearing, Settlement & Reconciliation.